

SWINBURNE UNIVERSITY OF TECHNOLOGY

DEPLOYMENT PORTFOLIO TASK 1

Pass to Distinction Implementation and Evidence Report

Student: Sidharth Krishnan

Student ID: 104089870

Unit: SWE40006 Software Deployment and Evolution

Teaching period: Semester 2, 2026

Submission date: 14 August 2026

Declared target: Task 1.3 - Distinction

HD task: Task 1.4 intentionally deferred; no HD claim is made in this report

Source status: Complete local Git-ready repository; public publication awaits student approval

Submission statement: Tasks 1.1, 1.2, and 1.3 were implemented, built, installed, executed, tested, and uninstalled on Windows. The Distinction MSI explicitly deploys three external runtime DLLs: Newtonsoft.Json, CsvHelper, and Humanizer.

Prepared from reproducible build logs and cursor-free evidence captured on the submission workstation.

1. Executive summary

This portfolio demonstrates an end-to-end Windows deployment workflow using WiX Toolset 4.0.6 and .NET 10. Task 1.1 packages a standard Hello World console application. Task 1.2 replaces the sample with HashGuard Desktop, a custom WinForms checksum utility. Task 1.3 extends that installer to include and verify multiple external DLL dependencies. Both MSIs use per-user installation paths so installation and clean removal can be demonstrated without requiring administrator rights.

Outcome: Final automated verification reported 4/4 tests passed. Both MSI projects built with 0 warnings and 0 errors. Silent install and uninstall operations returned exit code 0, and post-uninstall checks confirmed that installation directories, the HashGuard Start Menu shortcut, and uninstall registration were removed.

1.1 Toolchain

Component	Installed version	Role
Visual Studio	Community 2026 18.9.0	.NET desktop workload and solution/project authoring
.NET	SDK 10.0.400; Desktop Runtime 10.0.11	Build, test, publish, and execute the applications
WiX Toolset	CLI and SDK 4.0.6	Compile the two x64 MSI packages
HeatWave	Visual Studio extension installed	WiX project templates and Visual Studio integration
WingetCreate	1.12.13.0	Installed in preparation for the deferred HD publication task

1.2 Project structure

Project	Tier	Purpose
HelloWorld	Pass	Standard console application used to prove the basic WiX lifecycle
HelloWorldInstaller	Pass	Per-user x64 MSI for HelloWorld
HashGuard.Desktop	Credit	Custom WinForms checksum calculator and verifier
HashGuard.Tests	Credit	MSTest coverage for hashing, verification, JSON, and CSV
HashGuardInstaller	Distinction	WiX MSI with explicit external DLL components and Start Menu shortcut

1.3 Reproducibility controls

The WiX SDK and extension versions are pinned in the .wixproj files, NuGet dependency versions are locked, product and upgrade GUIDs are stable, and the evidence directory contains final test, build, install, execution, metadata, and uninstall logs. MSI SHA-256 hashes are recorded in evidence/33-msi-metadata.txt.

2. Task 1.1 - Pass: Hello World MSI

The Pass implementation is a small .NET console program that prints an assignment-specific banner. A WiX x64 package installs the published executable and its three companion runtime files to %LOCALAPPDATA%\Programs\SWE40006 Hello World. The MSI was installed silently, the installed executable was run from that directory, and the product was then removed by its MSI product code.

MSI	SWE40006-HelloWorld-1.0.0-x64.msi
Product version	1.0.0
Product code	{40C76FDB-606D-4DE6-9B17-F06149341B0A}
Install result	Exit code 0; four application files present
Execution result	Installed application printed the expected Task 1.1 banner
Uninstall result	Exit code 0; install directory removed

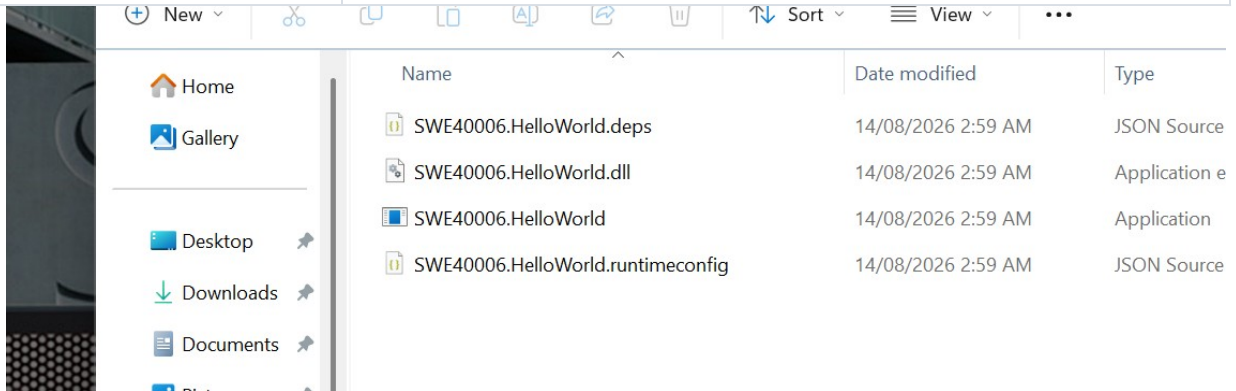


Figure 1. Task 1.1 installed directory containing the published Hello World executable and runtime files.

```

=====
SWE40006 Deployment Task 1.1
Hello World application is running.
=====
Executed successfully at 2026-08-14 02:59:41 +10:00

```

Figure 2. Console output from the installed Hello World executable.

3. Task 1.2 - Credit: HashGuard Desktop

HashGuard Desktop is an original C# WinForms application for calculating SHA-256 and SHA-512 file checksums. It accepts a file, calculates both hashes asynchronously, optionally compares an expected value, records a local audit history in JSON, and exports history to CSV. The interface identifies its external dependencies and exposes the complete hash values for copying and verification.

3.1 Application design

Layer	Implementation	Responsibility
Presentation	MainForm.cs	File selection, progress, hash display, comparison, history, and export actions
Hashing	HashService.cs	Buffered asynchronous SHA-256/SHA-512 computation using IncrementalHash
Persistence	HistoryStore.cs	Auditable local JSON history using Newtonsoft.Json
Export	CsvExportService.cs	CSV output through CsvHelper
Presentation helper	Humanizer	Readable dates and display text

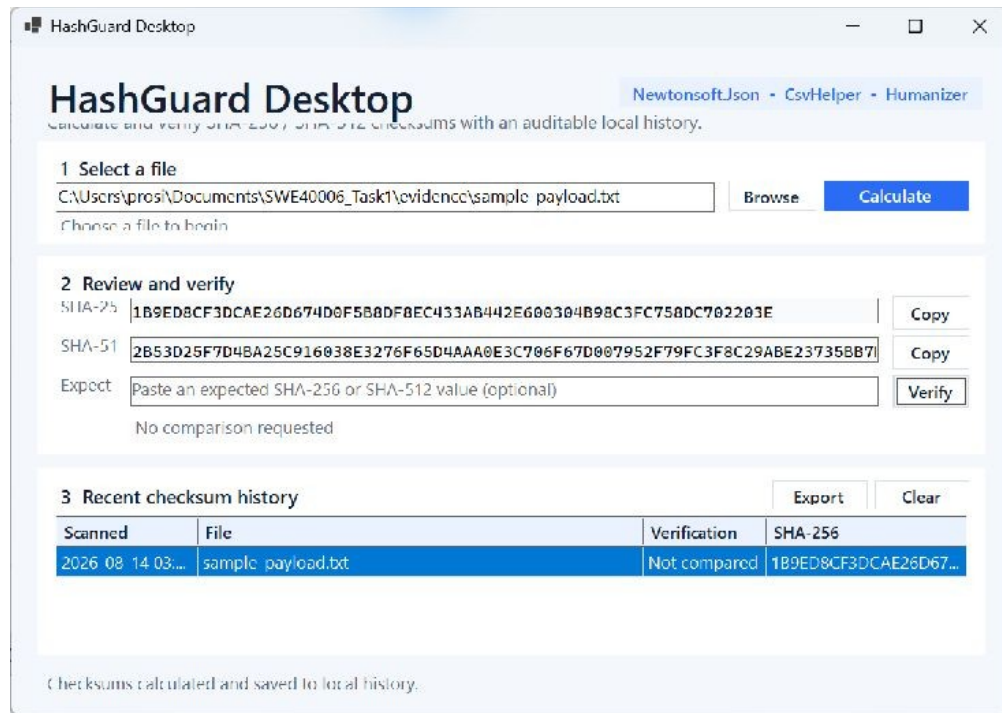


Figure 3. Cursor-free execution evidence: the installed HashGuard application calculated SHA-256 and SHA-512 values and stored a history record.

3.2 Core checksum implementation

C# excerpt - HashService.ComputeAsync

```
using var sha256 = IncrementalHash.CreateHash(HashAlgorithmName.SHA256);
using var sha512 = IncrementalHash.CreateHash(HashAlgorithmName.SHA512);
await using var stream = new FileStream(
    filePath,
    FileMode.Open,
    FileAccess.Read,
    FileShare.Read,
    bufferSize,
    FileOptions.Asynchronous | FileOptions.SequentialScan);

var buffer = new byte[bufferSize];
long totalRead = 0;
int bytesRead;

while ((bytesRead = await stream.ReadAsync(buffer, cancellationToken)) > 0)
{
    sha256.AppendData(buffer, 0, bytesRead);
    sha512.AppendData(buffer, 0, bytesRead);
    totalRead += bytesRead;
    progress?.Report(fileInfo.Length == 0 ? 100 : (int)(totalRead * 100 / fileInfo.Length));
}

progress?.Report(100);
return new HashResult(
    fileInfo.FullName,
    fileInfo.Length,
    Convert.ToString(sha256.GetHashAndReset()),
    Convert.ToString(sha512.GetHashAndReset()));
```

The implementation streams the file in 1 MiB buffers and appends each buffer to both hash algorithms. This avoids loading the entire file into memory and provides a progress callback for large inputs.

3.3 Automated verification

```
HashGuard.Desktop -> C:\Users\prosi\Documents\SWE40006_Task1\src\HashGuard.Desktop\bin\Release\net10.0-windows\HashGuard.Desktop.dll
HashGuard.Tests -> C:\Users\prosi\Documents\SWE40006_Task1\src\HashGuard.Tests\bin\Release\net10.0-windows\HashGuard.Tests.dll
Test run for C:\Users\prosi\Documents\SWE40006_Task1\src\HashGuard.Tests\bin\Release\net10.0-windows\HashGuard.Tests.dll (.NETCoreApp,Version=v10.0)
A total of 1 test files matched the specified pattern.

Passed! - Failed:    0, Passed:    4, Skipped:    0, Total:    4, Duration: 68 ms - HashGuard.Tests.dll (net10.0)
```

Figure 4. Final test run: 4 passed, 0 failed, 0 skipped.

Test	Expected behaviour	Result
Known hashes	SHA-256 and SHA-512 match known values	Passed
Mismatch verification	Malformed or different expected hash is rejected	Passed
JSON round trip	Newtonsoft.Json persists and restores a history record	Passed
CSV export	CsvHelper writes an export containing the history data	Passed

4. Task 1.3 - Distinction: external dependencies

Distinction criterion: The HashGuard MSI explicitly authors multiple runtime dependencies as individual WiX components. The final installation contains three external DLLs alongside the application: Newtonsoft.Json.dll, CsvHelper.dll, and Humanizer.dll.

4.1 Runtime dependency set

Dependency	Pinned version	Use in HashGuard
Newtonsoft.Json	13.0.4	Serializes and deserializes checksum history
CsvHelper	33.1.0	Exports the audit history to CSV
Humanizer.Core	3.0.10	Formats readable time and display values

4.2 WiX authoring

WiX v4 excerpt - explicit external DLL components

```
<Component Id="NewtonsoftJsonComponent" Guid="{9AAAAB83-2199-4AFB-B3F9-14AAA88A66A}">
  <File Id="Newtonsoft.Json.dll" Source="..\..\src\HashGuard.Desktop\bin\Release\net10.0-windows\win-
x64\publish\Newtonsoft.Json.dll" KeyPath="yes" />
</Component>
<Component Id="CsvHelperComponent" Guid="{F6EBD07F-2291-4527-9317-4739766DD50B}">
  <File Id="CsvHelper.dll" Source="..\..\src\HashGuard.Desktop\bin\Release\net10.0-windows\win-
x64\publish\CsvHelper.dll" KeyPath="yes" />
</Component>
<Component Id="HumanizerComponent" Guid="{EA9E4267-5AA4-4E47-9AAA-463335F372FD}">
  <File Id="Humanizer.dll" Source="..\..\src\HashGuard.Desktop\bin\Release\net10.0-windows\win-
x64\publish\Humanizer.dll" KeyPath="yes" />
</Component>
```

Every external file has its own stable component GUID and KeyPath. The package also defines the application executable, application DLL, dependency manifest, runtime configuration, a per-user installation directory, and a Start Menu shortcut.

```
<Directory Id="APPLICATIONPROGRAMSUSER" Name="HashGuard Desktop" />
</StandardDirectory>
</Fragment>

<Fragment>
  <ComponentGroup Id="HashGuardComponents" Directory="INSTALLFOLDER">
    <Component Id="HashGuardExeComponent" Guid="{F48A825E-434E-4095-8F37-5B12ABFC00AC}">
      <File Id="HashGuardExe" Source="..\..\src\HashGuard.Desktop\bin\Release\net10.0-windows\win-x64\publish\HashGuard.Desktop.exe" KeyPath="yes" />
    </Component>
    <Component Id="HashGuardDllComponent" Guid="{815918D3-C280-438F-8553-FC98BA71E1DC}">
      <File Id="HashGuard.dll" Source="..\..\src\HashGuard.Desktop\bin\Release\net10.0-windows\win-x64\publish\HashGuard.Desktop.dll" KeyPath="yes" />
    </Component>
    <Component Id="HashGuardDepsComponent" Guid="{D47537ED-5827-4176-8B14-6882DEFAC95}">
      <File Id="HashGuardDeps" Source="..\..\src\HashGuard.Desktop\bin\Release\net10.0-windows\win-x64\publish\HashGuard.Desktop.deps.json" KeyPath="yes" />
    </Component>
    <Component Id="HashGuardRuntimeComponent" Guid="{95DFE4D7-5DAB-418B-A235-75336D1E578F}">
      <File Id="HashGuardRuntime" Source="..\..\src\HashGuard.Desktop\bin\Release\net10.0-windows\win-x64\publish\HashGuard.Desktop.runtimeconfig.json" KeyPath="yes" />
    </Component>
    <Component Id="NewtonsoftJsonComponent" Guid="{9AAAAB83-2199-4AFB-B3F9-14AAA88A66A}">
      <File Id="Newtonsoft.Json.dll" Source="..\..\src\HashGuard.Desktop\bin\Release\net10.0-windows\win-x64\publish\Newtonsoft.Json.dll" KeyPath="yes" />
    </Component>
    <Component Id="CsvHelperComponent" Guid="{F6EBD07F-2291-4527-9317-4739766DD50B}">
      <File Id="CsvHelper.dll" Source="..\..\src\HashGuard.Desktop\bin\Release\net10.0-windows\win-x64\publish\CsvHelper.dll" KeyPath="yes" />
    </Component>
    <Component Id="HumanizerComponent" Guid="{EA9E4267-5AA4-4E47-9AAA-463335F372FD}">
      <File Id="Humanizer.dll" Source="..\..\src\HashGuard.Desktop\bin\Release\net10.0-windows\win-x64\publish\Humanizer.dll" KeyPath="yes" />
    </Component>
  </ComponentGroup>
</Fragment>

<Fragment>
  <Component Id="HashGuardShortcutComponent" Directory="APPLICATIONPROGRAMSUSER" Guid="{A9A3A31E-8556-439A-B3FF-8F66A0000000}">
```

Figure 5. Cursor-free browser rendering of the authored Package.wxs, including the three external DLL components.

4.3 Build and installed-file evidence

```
HashGuardInstaller -> C:\Users\prosi\Documents\SWE40006_Task1\installer\HashGuardInstaller\bin\Release\HashGuard-Desktop-1.0.0-x64.msi

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:00.27
```

Figure 6. Final HashGuard installer build: succeeded with 0 warnings and 0 errors.

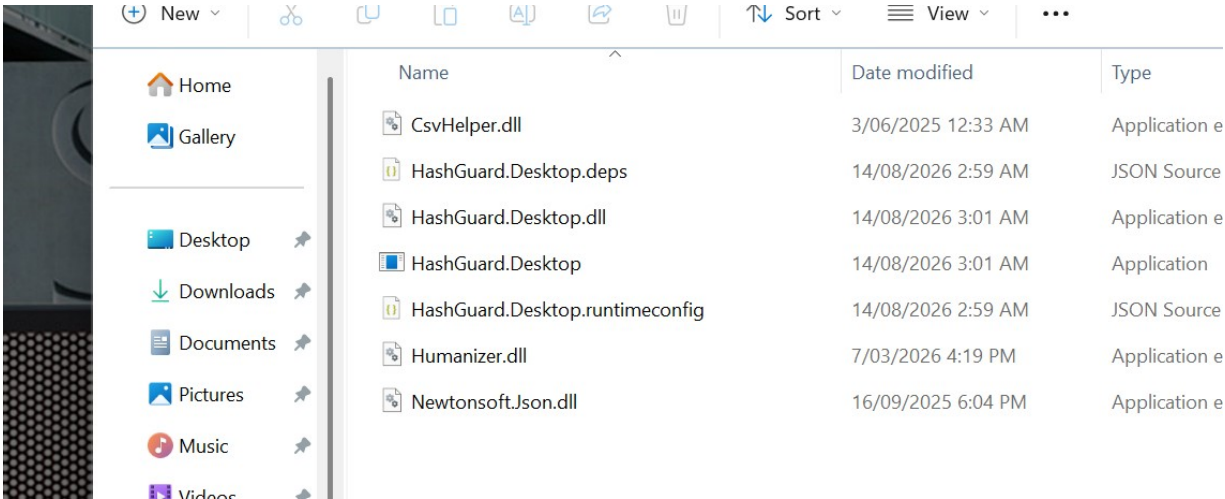


Figure 7. Installed HashGuard payload showing the application plus CsvHelper.dll, Humanizer.dll, and Newtonsoft.Json.dll.

4.4 MSI identity and lifecycle

MSI	HashGuard-Desktop-1.0.0-x64.msi
Product code	{C57B16A8-E9E5-4EE9-8020-EE069EB409A7}
Upgrade code	{23AB1A31-A44A-41FE-BBDA-5245030BAD76}
Size	1,093,632 bytes
SHA-256	8B44ED575B560DD0C996AEB5CC1CF06C06DE8C9A3F31D669A3C50134AB488ED1
Final install	Exit code 0; 7 files plus Start Menu shortcut
Execution	Installed application launched and processed evidence/sample-payload.txt
Final uninstall	Exit code 0; installation directory, shortcut, and ARP registration removed

5. Verification and evidence matrix

Requirement	Evidence	Verified result
Toolchain	27-toolchain.txt, 28-dotnet-tools.txt, 29-visual-studio-workload.json	.NET, WiX, HeatWave/Visual Studio workload recorded
Pass build	31-final-hello-msi-build.log	0 warnings; 0 errors
Pass install/run	06-hello-install.log, 07-hello-installed-files.txt, 08-hello-execution.log	Install and execution successful
Credit tests	30-final-tests.log	4 passed; 0 failed
Distinction build	32-final-hashguard-msi-build.log	0 warnings; 0 errors
Distinction install	19-hashguard-final-install.log, 20-hashguard-final-files.txt, 21-hashguard-arp.txt	Exit code 0; 3 external DLLs present
MSI metadata	33-msi-metadata.txt	Product identity and SHA-256 hashes recorded
Uninstall	40-hashguard-uninstall.log, 41-hello-uninstall.log, 42-post-uninstall-verification.txt	Both exit code 0; deployed artifacts removed

5.1 Manual checksum result

Input	evidence/sample-payload.txt
SHA-256	1B9ED8CF3DCAE26D674D0F5B8DF8EC433AB442E600304B98C3FC758DC702203E
SHA-512	2B53D25F7D4BA25C916038E3276F65D4AAA0E3C706F67D007952F79FC3F8C29AB E23735BB7D6E1A41C3AE4A89D9EE1694A82E1894C4A9EBC3F1B121A640C47AF
History	One record written to the local JSON audit history

5.2 Uninstall behaviour

The uninstallers remove only deployed program files, the HashGuard Start Menu folder, and Windows uninstall registration. The user's JSON history remains in %LOCALAPPDATA%\HashGuard Desktop\history.json by design, because user-created data should not be removed silently during application uninstall. Both MSI files remain in the repository build output and can reinstall the applications.

6. Troubleshooting and engineering decisions

6.1 WinForms designer serialization warning

Initial compilation raised WFO1000 because designer serialization interpreted public form state as design-time content. The form was refactored so application services and state are private readonly fields initialized in the constructor. This separated runtime state from designer serialization and restored a clean build.

6.2 WiX v4 UI migration

An initial WiX build used the WiX v3 UIRef pattern and failed with WIX0094 because that identifier is protected in WiX v4. The package was migrated to the WiX v4 UI extension namespace and now declares ui:WixUI with WixUI_InstallDir and INSTALLFOLDER. The WixToolset.UI.wixext version is pinned to 4.0.6.

6.3 Installed-app runtime failure

The first installed HashGuard build terminated before displaying its window. Windows Application Event Log evidence identified a .NET 10 NotSupportedException caused by a transparent button border setting. The custom button styling was corrected by explicitly setting BorderSize to 0. Tests, publish, MSI build, uninstall, reinstall, and application execution were then repeated successfully.

6.4 MSI validation policy

On the first non-elevated WiX build, ICE validation could not run because of workstation policy. Verification was strengthened with Windows Installer COM metadata inspection, SHA-256 hashes, real silent installations, installed-file and ARP checks, execution from the deployed directory, MSI log review, and complete uninstall cleanup checks. Final reproducibility builds completed with 0 warnings and 0 errors.

6.5 Packaging decisions

Decision	Reason	Effect
Per-user MSI	Avoids unnecessary elevation for a student utility	Installs under LocalAppData and supports silent testing
Explicit DLL components	Makes Distinction dependency evidence unambiguous	Each external DLL has a stable component GUID and key path
Framework-dependent publish	Keeps the MSI small while targeting the installed Desktop Runtime	Runtimeconfig and deps files are deployed with the application
Retain user history	Avoid unexpected destruction of user-created data	Program files are removed while history.json remains

7. Reproduction procedure

Run the following commands from the repository root on Windows with .NET 10 and WiX Toolset 4.0.6 available. The projects use pinned package versions, so restore and build results are reproducible.

PowerShell - build and test

```
dotnet test src/HashGuard.Tests/HashGuard.Tests.csproj -c Release
dotnet publish src/HashGuard.Desktop/HashGuard.Desktop.csproj -c Release -r win-x64 --self-contained false
dotnet build installer/HelloWorldInstaller/HelloWorldInstaller.wixproj -c Release
dotnet build installer/HashGuardInstaller/HashGuardInstaller.wixproj -c Release
```

PowerShell - install and uninstall

```
msiexec /i installer\HelloWorldInstaller\bin\Release\SWE40006-HelloWorld-1.0.0-x64.msi /qn
msiexec /i installer\HashGuardInstaller\bin\Release\HashGuard-Desktop-1.0.0-x64.msi /qn
msiexec /x {40C76FDB-606D-4DE6-9B17-F06149341B0A} /qn
msiexec /x {C57B16A8-E9E5-4EE9-8020-EE069EB409A7} /qn
```

7.1 Expected outputs

Step	Success condition	Final observed result
Test	All automated tests pass	4/4 passed
Build	Both MSI projects succeed	0 warnings; 0 errors
Install	MSI exits 0 and files appear	Both exits 0; 4 and 7 files respectively
Run	Installed executable starts	Hello banner and HashGuard checksum result captured
Uninstall	MSI exits 0 and deployed artifacts disappear	Both exits 0; directories and shortcut removed

8. HD task boundary and next plan

Scope boundary: Task 1.4 is not claimed. The MSI and evidence are ready for publication work, but GitHub Release creation and WinGet or Chocolatey publication are intentionally deferred until the next stage, as requested.

The recommended HD route is a GitHub Release plus a valid WinGet manifest submission. The release must host the immutable x64 MSI, the repository must contain the license and source, the manifest must use the recorded installer URL and SHA-256 value, and validation should be performed with WingetCreate before opening a pull request to microsoft/winget-pkgs. This plan is detailed in docs/HD-PLAN.md.

HD stage	Planned action	Evidence to retain
Release	Publish v1.0.0 and attach HashGuard-Desktop-1.0.0-x64.msi	Release URL, asset hash, license, and release notes
Manifest	Generate multi-file WinGet manifest with WingetCreate	Installer URL, architecture, product code, and SHA-256
Validation	Run local and sandbox manifest validation	Command output and successful install/uninstall
Submission	Open a manifest PR to microsoft/winget-pkgs	PR URL and validation status

9. References

FireGiant. HeatWave Community Edition and Visual Studio integration. <https://docs.firegiant.com/heatwave/>

FireGiant. WiX Toolset v4 UI extension (WixUI). <https://docs.firegiant.com/wix/tools/wixext/wixui/>

Microsoft. IncrementalHash class. <https://learn.microsoft.com/dotnet/api/system.security.cryptography.incrementalhash>

NuGet. Newtonsoft.Json 13.0.4. <https://www.nuget.org/packages/Newtonsoft.Json/13.0.4>

NuGet. CsvHelper 33.1.0. <https://www.nuget.org/packages/CsvHelper/33.1.0>

NuGet. Humanizer.Core 3.0.10. <https://www.nuget.org/packages/Humanizer.Core/3.0.10>

Microsoft. Submit packages to the Windows Package Manager repository. <https://learn.microsoft.com/windows/package-manager/package/repository>

Microsoft. winget-pkgs community repository. <https://github.com/microsoft/winget-pkgs>

10. Integrity statement

All screenshots and logs in this report were produced from the accompanying source and MSI artifacts on the submission workstation. The final evidence images are cursor-free. No HD publication outcome is represented as complete. Product identities, hashes, and command results can be independently reproduced from the repository.